

Упражнения към Лекция 1: Въведение в AI и ML

Задачите по-долу затвърждават основните концепции от лекцията: bigram модели, генериране на текст, перплексия и представяне на думи. Решават се с химикал и лист хартия. **Тези задачи не са задължителни, но са полезна подготовка към контролните.**

Упражнение 1: Bigram вероятности

Даден е мини-корпус от 13 символа (включително интервалите):

```
а б а б а а б
```

Речникът е $V = \{ \text{а}, \text{б}, \text{``} \}$.

Задачи:

- 1. Преброете всички bigram-и (двойки последователни символи) и попълнете таблицата с бройки:

Текущ ↓ \ Следващ →	а	б	``
а			
б			
``			

- 2. Нормализирайте всеки ред — разделете всеки елемент на сумата по реда, за да получите $P(\text{следващ} \mid \text{текущ})$.
- 3. Изчислете вероятността на последователността "б а" (3 символа): $P(\mid \text{`б`}) \cdot P(\text{`а`} \mid)$.
- 4. Изчислете вероятността на "б б" . Какъв проблем възниква? Как Лапласово изглаждане (добавяне на $\alpha = 1$ към всяка клетка преди нормализация) би го решило?

Упражнение 2: Генериране от bigram модел

Имаме bigram модел с таблица от вероятности:

Текущ ↓ \ Следващ →	а	д	н	``
а	0.05	0.15	0.20	0.60
д	0.65	0.05	0.10	0.20
н	0.55	0.05	0.05	0.35
``	0.10	0.25	0.40	0.25

Генерираме текст, започвайки от `` (интервал).

Задачи:

- 1. При greedy decoding (винаги избираме най-вероятния символ) — какви са първите 8 символа? Получава ли се смислен текст?

- Изчислете вероятността на последователността " дана" (интервал + дана): $P(d | \text{' '}) \cdot P(a | d) \cdot P(n | a) \cdot P(a | n)$.
- Може ли този модел да генерира думата "зная" ? Защо?
- По таблицата: след `` най-вероятен е н , а след н най-вероятен е а . Greedy ще генерира " на на на..." . Какво ни казва това за greedy decoding като стратегия?

Упражнение 3: Перплексия — интуиция и граници

Перплексията измерва колко "изненадан" е моделът от текста. При речник от V символа:

- Случаен модел (равномерно разпределение) има перплексия $= V$.
- Перфектен модел, който винаги знае следващия символ, има перплексия $= 1$.

Задачи:

- Bigram моделът от лекцията има перплексия ≈ 12 при $V = 197$. Интуитивно какво означава числото 12 — между колко символа ефективно "избира" моделът на всяка стъпка?
- Перплексията е дълго време основната метрика за езикови модели. Защо? Какво я прави полезна от, например, точност (accuracy)?
- Защо перплексията се ползва по-рядко днес?
- Защо не можем директно да сравним перплексията на character-level модел с тази на GPT-2?

Упражнение 4: Защо n-gram моделите не са достатъчни

Разгледайте изречението:

"Царят, който управляваше седемте кралства в продължение на много години, накрая ____."

Искаме да предскажем последната дума.

Задачи:

- Bigram модел на ниво думи вижда само предходната дума. Коя е тя? Достатъчна ли е за смислено предсказване?
- Думата "Царят" е на 12 позиции разстояние. Какъв n (в n -gram модел) е нужен, за да я "види"? Колко записа би съдържала таблицата, ако речникът е 50 000 думи?
- N -gram моделът третира всяка комбинация от думи като отделен запис. Ако моделът е видял "кралят... накрая абдикира", но не е видял "царят... накрая ____" — ще може ли да прехвърли знанието? Защо не?
- Формулирайте с едно изречение фундаменталната разлика между "таблица с бройки" (n -gram модел) и "функция с параметри" (невронна мрежа) по отношение на генерализацията.

Упражнение 5: Embeddings и представяне на думите

В n -gram модела всеки символ (или дума) е индекс: "котка" = 7, "куче" = 42, "маса" = 103. Тези индекси не носят никаква информация за значението.

Алтернативата е **embedding** — всяка дума се представя като кратък плътен вектор (напр. 2D):

Дума	Вектор
котка	(0.9, 0.2)

куче	(0.8, 0.3)
риба	(0.6, 0.1)
маса	(0.1, 0.8)
стол	(0.2, 0.9)

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

Задачи:

1. Изчислете $\cos(\text{котка}, \text{куче})$ и $\cos(\text{котка}, \text{маса})$. Коя двойка е по-сходна по модела?
2. "Крал" и "цар" означават почти едно и също. Ако моделът е видял много текст с "крал", но малко с "цар" — как embedding представянето помага на модела да се справи по-добре с "цар"?
3. GPT-2 има embedding матрица $50\,257 \times 768$. Колко числа е това? При 124 милиона параметъра общо, какъв процент от модела е само в embedding слоя?

Забележка: Не всички embeddings кодират семантично сходство. Embedding слой на GPT-2 е оптимизиран за предвиждане на следващ токен, не за сходство между думи — затова при демото от лекцията "king" и "banana" имаха подобно сходство на "king" и "throne". Модели като Word2Vec или Sentence-BERT са специално обучени за семантична близост.

Упражнение 6: От n-грам до невронна мрежа

Обобщаваща задача.

Задачи:

1. Напишете няколко предимства на n-грам моделите. Напишете няколко и от техните недостатъци.